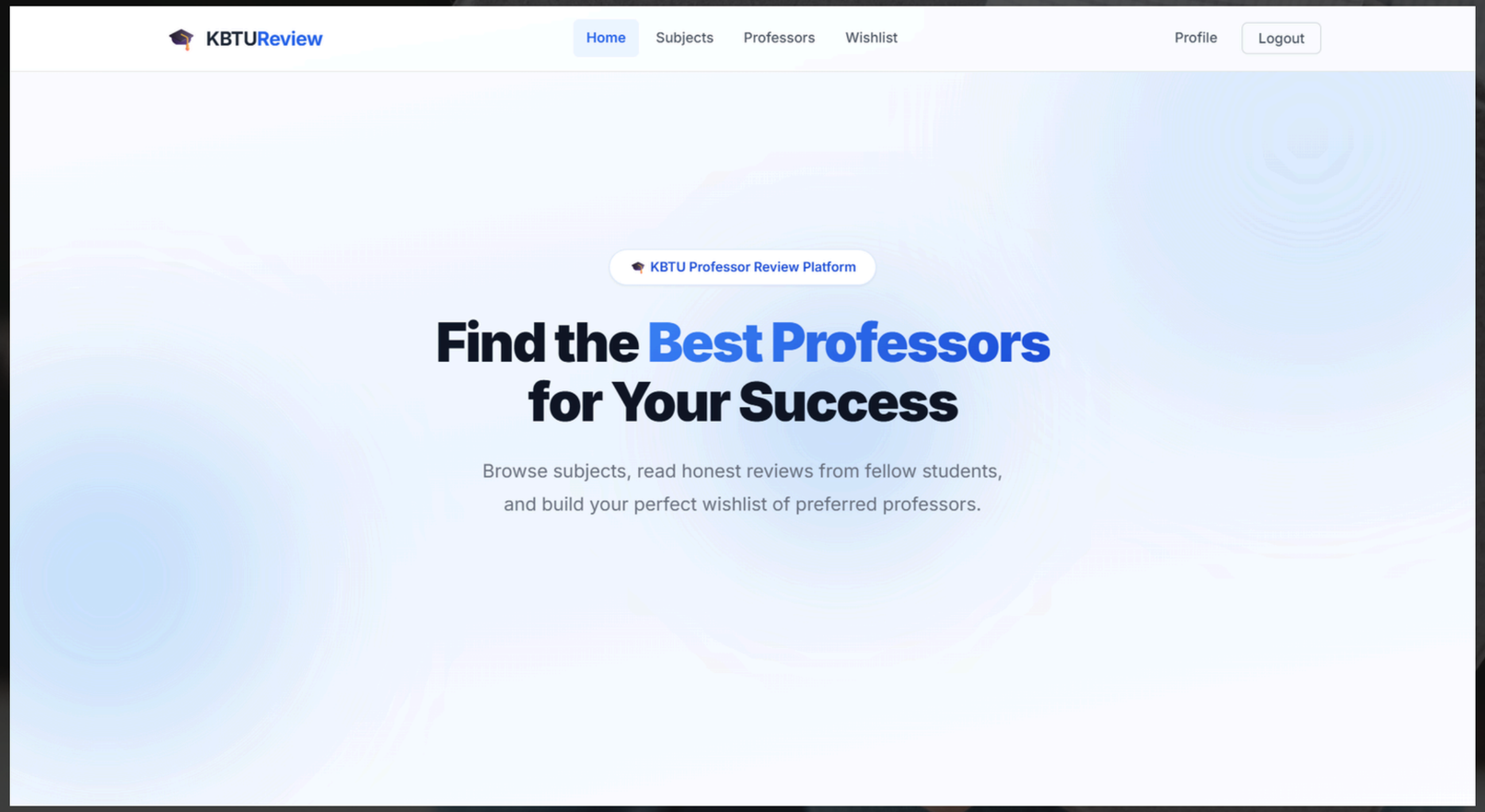
KBTU Professor Review Platform

Team: Bolotnov Konstantin · Kabirov Ravil · Abdrakhmanov Elmurad

Choosing the right professor is one of the most important decisions a student makes each semester — yet there's rarely a reliable place to get honest feedback. Our platform gives KBTU students a convenient and transparent environment to browse academic subjects, explore professors teaching those subjects, and read or share real reviews based on rating, difficulty, and personal experience.



BACKEND

In backend we are covered all requirements given.

```python
        fields = '__all__'

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ['rev_id', 'user', 'professor', 'subject', 'rating', 'di

class SubjectSerializer(serializers.ModelSerializer):
    class Meta:
        model = Subject
        fields = ['subj_id', 'name', 'description']


class LoginSerializer(serializers.Serializer):
    username = serializers.CharField()
    password = serializers.CharField()

    def validate(self, data):
        username = data['username']
        password = data['password']
        user = User.objects.filter(username=username).first()
        if not user:
            raise serializers.ValidationError("User not found")
```

Serializers for All of Models. Also 2 serializers for login and register.

```python
@api_view(['GET'])
def get_reviews_by_professor(request, prof_id):
    professor = Professor.objects.filter(prof_id=prof_id).first()

    if not professor:
        return Response({"error": "Professor not found"}, status=404)

    reviews = Review.objects.reviewsOfProfessor(professor)

    review_type = request.query_params.get('type')

    if review_type == 'positive':
        reviews = reviews.filter(rating__gte=4.0)
    elif review_type == 'negative':
        reviews = reviews.filter(rating__lte=2.5)

    serializer = ReviewSerializer(reviews, many=True)
    return Response(serializer.data)

class ReviewAPIView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        reviews = Review.objects.all()
        serializer = ReviewSerializer(reviews, many=True)
        return Response(serializer.data)
    def post(self, request):
```

All of the required types of views: CBV and FBV

```python
class User(AbstractUser):
    username = models.CharField(max_length=30, unique=True)
    role = models.CharField(max_length=20, choices=Role.choices, default=Role.STUDENT)
    course = models.FloatField(default=1, validators=[
        MaxValueValidator(4.0),
        MinValueValidator(1.0)
    ])
    school = models.CharField(max_length=10, choices=School.choices)

class Professor(models.Model):
    prof_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=200)
    description = models.TextField()
    subjects = models.ManyToManyField('Subject', related_name='professors')

class Subject(models.Model):
    subj_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)
    description = models.TextField()

class ReviewManager(models.Manager):
    def reviewsOfProfessor(self, prof):
        return self.filter(professor=prof)
```

User model for authentication. Models to contain entities for frontend. Custom manager for Reviews.

FRONTEND

In frontend we are covered all requirements given.

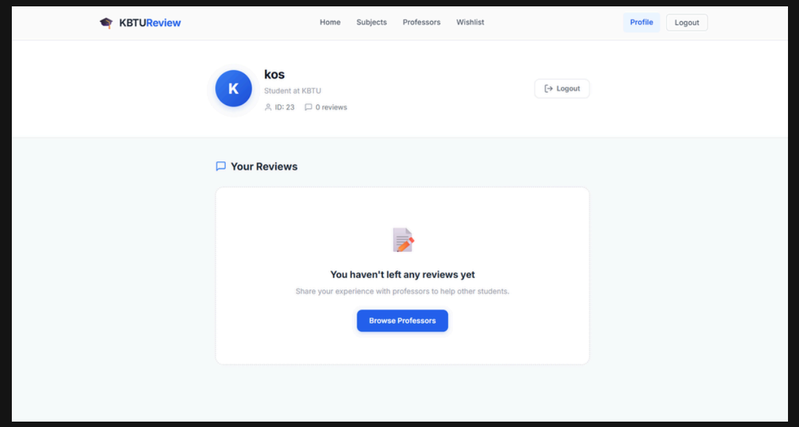
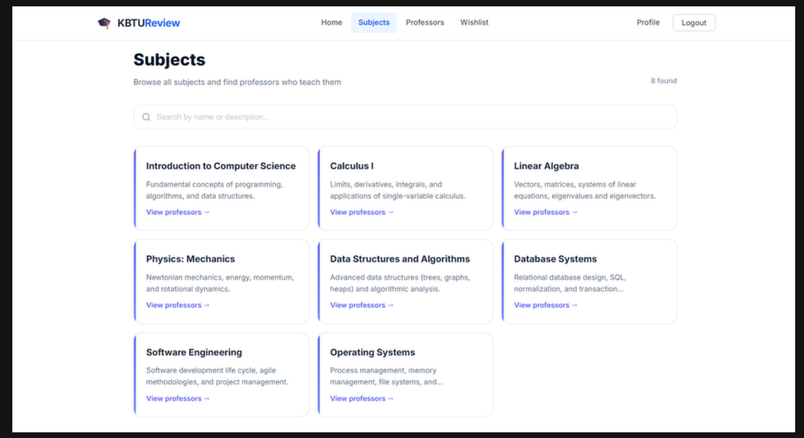
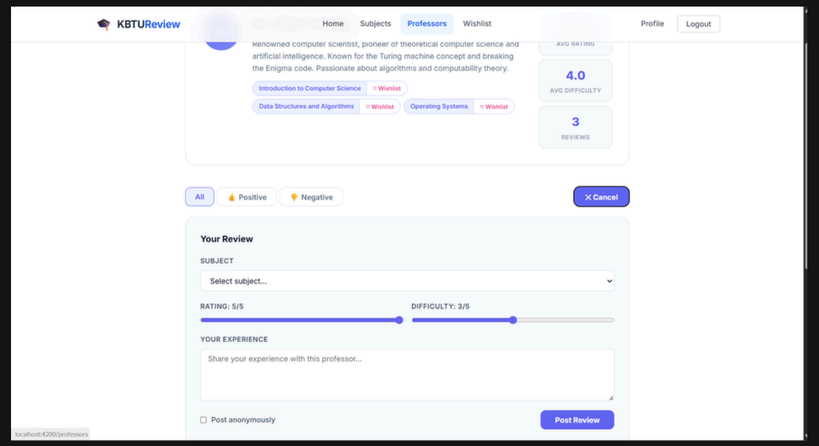
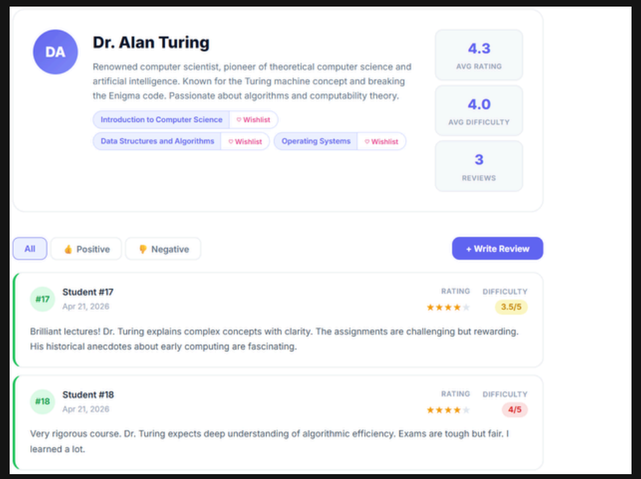
```
import { Routes } from '@angular/router';
import { authGuard } from './core/guards/auth-guard';

export const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },

  { path: 'login', loadComponent: () => import('./features/auth/pages/login/login').then(m => m.Login) },
  { path: 'register', loadComponent: () => import('./features/auth/pages/register/register').then(m => m.Register) },

  { path: 'home', loadComponent: () => import('./features/home/pages/home-page/home-page').then(m => m.HomePage) },

  { path: 'subjects', loadComponent: () => import('./features/subjects/pages/subjects-list/subjects-list').then(m => m.
  { path: 'subjects/:id', loadComponent: () => import('./features/subjects/pages/subject-detail/subject-detail').then(m

  { path: 'professors', loadComponent: () => import('./features/professors/pages/professors-list/professors-list').then
  { path: 'professors/:id', loadComponent: () => import('./features/professors/pages/professor-detail/professor-detail'

  {
    path: '',
    canActivate: [authGuard],
    children: [
      { path: 'wishlist', loadComponent: () => import('./features/wishlist/pages/wishlist-page/wishlist-page').then(m =>
      { path: 'profile', loadComponent: () => import('./features/profile/pages/profile-page/profile-page').then(m => m.
    ]
  },

  { path: '**', loadComponent: () => import('./features/not-found/pages/not-found-page/not-found-page').then(m => m.Noti
];
```

Routing module with authguard implementation and lazy-loading.

```
const ACCESS_TOKEN_KEY = 'access_token';
const REFRESH_TOKEN_KEY = 'refresh_token';

function addAuthHeader(req: HttpRequest<unknown>, token: string) {
  return req.clone({
    setHeaders: { Authorization: `Bearer ${token}` }
  });
}

export const authInterceptor: HttpInterceptorFn = (
  req: HttpRequest<unknown>,
  next: HttpHandlerFn
) => {
  const authService = inject(AuthService);
  const accessToken = localStorage.getItem(ACCESS_TOKEN_KEY);


  const isAuthEndpoint = req.url.includes('/login/') || req.url.includes('/register/') || req.url.includes('/refresh/');

  if (!accessToken || isAuthEndpoint) {
    return next(req);
  }

  const authReq = addAuthHeader(req, accessToken);
```

There is http interceptor to catch http requests and add header to them

```
models
  professor.model.ts
  review.model.ts
  subject.model.ts
  user.model.ts
services
  auth.spec.ts
  auth.ts
  professor.spec.ts
  professor.ts
  review.spec.ts
  review.ts
  subject.spec.ts
  subject.ts
  user.spec.ts
  user.ts
  wishlist.spec.ts
  wishlist.ts
```

There is interfaces and services to interact with back-end APIs

Functionality